Iniciación práctica al análisis de datos OMOP

Estudios de caracterización (PatientProfiles and CohortCharacteristics)

Anna Palomar

apalomar@idiapjgol.org

Marzo 2026

Introducción

A la hora de hacer un estudio, una vez tenemos la población definida, ¿cuál es el **primer paso** que debemos hacer?

Introducción

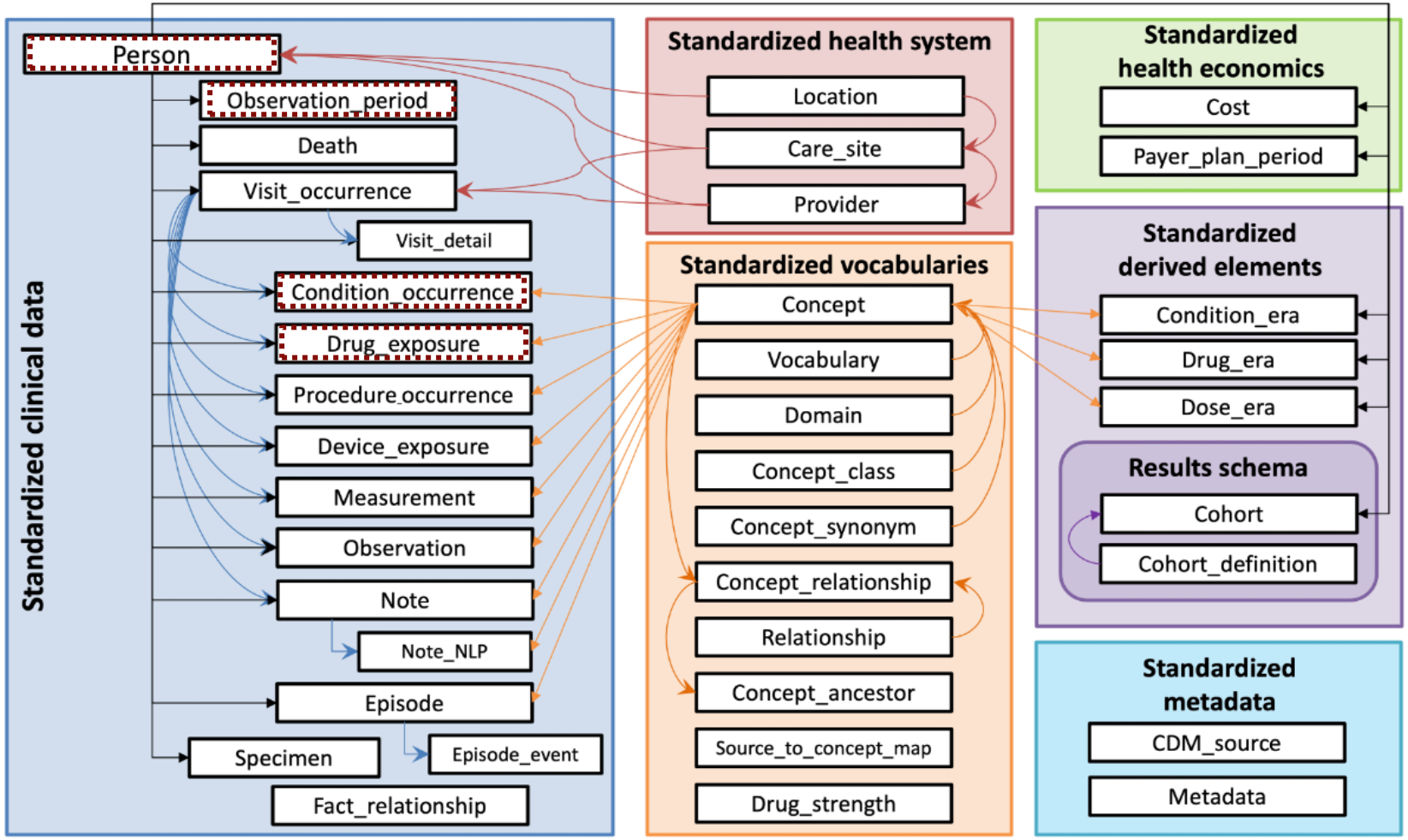
A la hora de hacer un estudio, una vez tenemos la población definida, ¿cuál es el **primer paso** que debemos hacer?

Hacer una **descriptiva** de nuestra **población de estudio**: Características demográficas, condiciones previas, historia previa

- Tabla 1

Antes

Calcular edad, calcular historia previa, juntar tablas.



Antes

Calcular edad, calcular historia previa, juntar tablas.

Pueden ser **muchas líneas de código**...

Ahora

Iniciación práctica al análisis de datos OMOP

PatientProfiles

25ANYS IDIAP JORDI GOL

Objetivo

- El objetivo del paquete ***PatientProfiles*** es **simplificar el código** necesario para **caracterizar individuos** (características demográficas, historia previa, etc.)

PatientProfiles

- Se ha desarrollado para el **Proyecto de Darwin EU®** por el equipo OxInfer.

- Está disponible en CRAN y se puede instalar fácilmente en R.

- ¿Cómo podemos **descargarlo**?

```
install.packages("PatientProfiles")
library(PatientProfiles)
```

https://cran.r-project.org/web/packages/PatientProfiles/index.html

PatientProfiles

Contiene un **conjunto de funciones** que permiten **añadir las características individuales** más comunes en **cualquier tabla del OMOP CDM** que contiene datos a **nivel de paciente** (por ejemplo, aparición de enfermedades, exposición a fármacos, etc.)

PatientProfiles

Contiene un conjunto de funciones que permiten añadir las características individuales más comunes en cualquier tabla del OMOP CDM que contiene datos a nivel de paciente (por ejemplo, aparición de enfermedades, exposición a fármacos, etc.)

¿Cuáles son estas **tablas**?

PatientProfiles

Contiene un conjunto de funciones que permiten añadir las características individuales más comunes en cualquier tabla del OMOP CDM que contiene datos a nivel de paciente (por ejemplo, aparición de enfermedades, exposición a fármacos, etc.)

¿Cuáles son estas **tablas**?
Person, conditions, drug_exposures, observation_period...

PatientProfiles

Contiene un conjunto de funciones que permiten añadir las características individuales más comunes en cualquier tabla del OMOP CDM que contiene datos a nivel de paciente (por ejemplo, aparición de enfermedades, exposición a fármacos, etc.)

¿Cuáles son estas tablas?
Person, conditions, drug_exposures, observation_period...

¿Cuáles son las **características** que queremos añadir?

PatientProfiles

Contiene un conjunto de funciones que permiten añadir las características individuales más comunes en cualquier tabla del OMOP CDM que contiene datos a nivel de paciente (por ejemplo, aparición de enfermedades, exposición a fármacos, etc.)

¿Cuáles son estas tablas?
Person, conditions, drug_exposures, observation_period...

¿Cuáles son las **características** que queremos añadir?

Edad, sexo, historia previa, intersecciones entre cohortes...

PatientProfiles

PatientProfiles 1.5.0 Reference Articles Changelog Search

PatientProfiles
Adding patient demographics
Adding cohort intersections
Adding concept intersections
Adding table intersections
Summarise result

PatientProfiles

CRAN 1.5.0 R-CMD-check passing lifec ownloads 42K

Package overview

PatientProfiles contains functions for adding characteristics to OMOP CDM tables containing patient level data (e.g. condition occurrence, drug exposure, and so on) and OMOP CDM cohort tables. The characteristics that can be added include an individual´s sex, age, and days of prior observation Time varying characteristics, such as age, can be estimated relative to any date in the corresponding table. In addition, PatientProfiles also provides functionality for identifying intersections between a cohort table and OMOP CDM tables containing patient level data or other cohort tables.

Package installation

You can install the latest version of PatientProfiles like so:

```
install.packages("PatientProfiles")
```

Links

View on CRAN

Browse source code

Report a bug

License

Full license

Apache License (>= 2)

Community

Contributing guide

Citation

Citing PatientProfiles

Developers

Martí Català
Author, maintainer

Yuchen Guo
Author

https://darwin-eu.github.io/PatientProfiles/

PatientProfiles

PatientProfiles 1.5.0 Reference Articles Changelog Search

PatientProfiles
Adding patient demographics
Adding cohort intersections
Adding concept intersections
Adding table intersections
Summarise result

PatientProfiles

CRAN 1.5.0 R-CMD-check passing lifec ownloads 42K

Package overview

PatientProfiles contains functions for adding characteristics to OMOP CDM tables containing patient level data (e.g. condition occurrence, drug exposure, and so on) and OMOP CDM cohort tables. The characteristics that can be added include an individual´s sex, age, and days of prior observation Time varying characteristics, such as age, can be estimated relative to any date in the corresponding table. In addition, PatientProfiles also provides functionality for identifying intersections between a cohort table and OMOP CDM tables containing patient level data or other cohort tables.

Package installation

You can install the latest version of PatientProfiles like so:

```
install.packages("PatientProfiles")
```

Links

View on CRAN

Browse source code

Report a bug

License

Full license

Apache License (>= 2)

Community

Contributing guide

Citation

Citing PatientProfiles

Developers

Martí Català

Author, maintainer

Yuchen Guo

Author

https://darwin-eu.github.io/PatientProfiles/

addAge()

Si queremos **añadir la edad** a cualquier tabla que contenga la información a nivel del paciente (tablas en el cdm, cohortes creadas).

```
cdm$curso_omop_covid <- cdm$curso_omop_covid |>
  addAge()
```

addAge()

Si queremos añadir la edad a cualquier tabla que contenga la información a nivel del paciente (tablas en el cdm, cohortes creadas).

```
cdm$curso_omop_covid <- cdm$curso_omop_covid |>
  addAge()
```

¿Qué **problemas** pueden surgir a la hora de calcular la edad?

addAge()

Si queremos añadir la edad a cualquier tabla que contenga la información a nivel del paciente (tablas en el cdm, cohortes creadas).

```
cdm$curso_omop_covid <- cdm$curso_omop_covid |>
  addAge()
```

¿Qué **problemas** pueden surgir a la hora de calcular la edad?

Puede ser que nos falte información sobre el mes o el día de nacimiento.

addAge()

```
addAge(
  x,
  indexDate = "cohort_start_date",
  ageName = "age",
  ageGroup = NULL,
  ageMissingMonth = 1,
  ageMissingDay = 1,
  ageImposeMonth = FALSE,
  ageImposeDay = FALSE,
  ageUnit = "years",
  missingAgeGroupValue = "None",
  name = NULL
)
```

Tabla con individuos en el **cdm**

Variable en x que contiene la **data** en que computar la edad

Nombre de la nueva columna que contiene edad

Listado de **grupos** de edad a añadir

Mes y día del año a asignar a los individuos que no tienen esta información. Por defecto:1

Si queremos considerar que faltan el mes y día del año de nacimiento para todos los individuos.

Unidades en años, meses o días

Valor para incluir si falta la edad

Nombre de la **nueva tabla**; si es NULL, se devuelve una tabla temporal.

addAge()

```
> cdm$covid
# Source:   table<covid> [?? x 4]
# Database: DuckDB v1.3.2 [apalomar@Windows 10 x64:R 4.4.1/C:\Users\apalomar\AppData\Local\
SUzkQg\file5dc44dd01bd.duckdb]
   cohort_definition_id subject_id cohort_start_date cohort_end_date
                  <int>      <int> <date>            <date>
 1                    1       8441 2020-08-09        2020-09-02
 2                    1       2002 2021-04-23        2021-04-23
 3                    1       4029 2020-08-27        2020-09-20
 4                    1       4677 2020-07-31        2020-08-14
 5                    1       7852 2020-12-05        2020-12-27
 6                    1       5795 2021-03-31        2021-04-14
 7                    1       6448 2021-03-01        2021-03-01
 8                    1        157 2020-11-08        2020-11-29
 9                    1       4285 2020-11-13        2020-12-20
10                    1      10585 2021-02-23        2021-03-23
# ℹ more rows
# ℹ Use `print(n = ...)` to see more rows
```

addAge()

```
> cdm$covid |> addAge()
# Source:   table<og_009_1772701727> [?? x 5]
# Database: DuckDB v1.3.2 [apalomar@Windows 10 x64:R 4.4.1/C:\Users\apalomar\AppData\Local\
SUzkQg\file5dc44dd01bd.duckdb]
   cohort_definition_id subject_id cohort_start_date cohort_end_date  age
                  <int>      <int> <date>            <date>          <int>
 1                    1          8 2021-01-08        2021-02-13        2
 2                    1         36 2020-10-09        2020-11-07        4
 3                    1         47 2020-11-26        2020-12-25       29
 4                    1         52 2020-12-08        2021-01-08       55
 5                    1         58 2020-07-12        2020-08-02       53
 6                    1         72 2020-12-02        2021-01-06       76
 7                    1         89 2020-11-27        2020-12-13       20
 8                    1        106 2020-08-16        2020-09-20       90
 9                    1        137 2020-09-08        2020-09-30        7
10                    1        156 2021-01-02        2021-02-08       46
# i more rows
# i Use `print(n = ...)` to see more rows
```

addAge()

También se pueden crear **grupos de edades** y darles un nombre

```
cdm$covid_age <- cdm$covid |>
  addAge(ageGroup = list(c(0,18), c(19,45), c(46,80), c(81, 110)),
         name="covid_age")
```

addAge()

También se pueden crear **grupos de edades** y darles un nombre

```
> cdm$covid_age <- cdm$covid |>
+   addAge(ageGroup = list(c(0,18), c(19,45), c(46,80), c(81, 110)),
+          name="covid_age")
> cdm$covid_age
# Source:   table<covid_age> [?? x 6]
# Database: DuckDB v1.3.2 [apalomar@Windows 10 x64:R 4.4.1/C:\Users\apalomar\AppData\Local
SUzkQg\file5dc44dd01bd.duckdb]
   cohort_definition_id subject_id cohort_start_date cohort_end_date   age age_group
                  <int>      <int> <date>            <date>          <int> <chr>
 1                    1          8 2021-01-08        2021-02-13          2 0 to 18
 2                    1         36 2020-10-09        2020-11-07          4 0 to 18
 3                    1         47 2020-11-26        2020-12-25         29 19 to 45
 4                    1         52 2020-12-08        2021-01-08         55 46 to 80
 5                    1         58 2020-07-12        2020-08-02         53 46 to 80
 6                    1         72 2020-12-02        2021-01-06         76 46 to 80
 7                    1         89 2020-11-27        2020-12-13         20 19 to 45
 8                    1        106 2020-08-16        2020-09-20         90 81 to 110
 9                    1        137 2020-09-08        2020-09-30          7 0 to 18
10                    1        156 2021-01-02        2021-02-08         46 46 to 80
# i more rows
# i Use `print(n = ...)` to see more rows
```

addSex()

De forma parecida, podemos añadir **información sobre el sexo** de cada individuo.

```
addSex(
  x,
  sexName = "sex",
  missingSexValue = "None",
  name = NULL
)
```

- `x` → Tabla con individuos en el **cdm**
- `sexName` → **Nombre** de la nueva columna que contiene el sexo
- `missingSexValue` → **Valor para incluir** si falta el sexo

addSex()

De forma parecida, podemos añadir **información sobre el sexo** de cada individuo.

```
> cdm$covid |> addSex()
# Source:   table<og_010_1772701847> [?? x 5]
# Database: DuckDB v1.3.2 [apalomar@Windows 10 x64:R 4.4.1/C:\Users\apalomar\AppData\Local\T
SUzkQg\file5dc44dd01bd.duckdb]
   cohort_definition_id subject_id cohort_start_date cohort_end_date sex
                  <int>      <int> <date>            <date>          <chr>
 1                    1          8 2021-01-08        2021-02-13      Male
 2                    1         36 2020-10-09        2020-11-07      Male
 3                    1         47 2020-11-26        2020-12-25      Female
 4                    1         52 2020-12-08        2021-01-08      Female
 5                    1         58 2020-07-12        2020-08-02      Female
 6                    1         72 2020-12-02        2021-01-06      Male
 7                    1         89 2020-11-27        2020-12-13      Male
 8                    1        106 2020-08-16        2020-09-20      Male
 9                    1        137 2020-09-08        2020-09-30      Male
10                    1        156 2021-01-02        2021-02-08      Male
# i more rows
# i Use `print(n = ...)` to see more rows
```

addPriorObservation()

Podemos añadir la **historia previa (días o fecha)** en base al periodo de observación correspondiente.

Por ejemplo, podemos añadir la historia previa y después filtrar a los pacientes que tengan al menos un año de historia previa.

addPriorObservation()

Podemos añadir la **historia previa (días o fecha)** en base al periodo de observación correspondiente.

```
addPriorObservation(
  x,
  indexDate = "cohort_start_date",
  priorObservationName = "prior_observation",
  priorObservationType = "days",
  name = NULL
)
```

- Tabla con individuos en el **cdm**
- Variable en x que contiene la **data** en que computar la observación previa
- **Nombre** de la nueva columna a añadir
- Indicar si queremos **días o en fecha**
- **Nombre** de la nueva tabla, si NULL se devuelve una tabla temporal

addPriorObservation()

Podemos añadir la **historia previa (días)** en base al periodo de observación correspondiente.

```
> cdm$curso_omop_covid |> addPriorObservation(priorObservationType = "days")
# Source:   table<og_004_1741339394> [?? x 5]
# Database: DuckDB v1.1.3 [apalomar@Windows 10 x64:R 4.4.1/C:\Users\apalomar\AppData\Local
\Temp\RtmpSsdhuU\file1e5c18635962.duckdb]
   cohort_definition_id subject_id cohort_start_date cohort_end_date prior_observation
                  <int>      <dbl> <date>            <date>                       <int>
 1                    1        531 2021-09-07        2021-09-17                    2895
 2                    1       6285 2020-11-04        2020-11-14                    2509
 3                    1       8096 2021-01-07        2021-01-17                    2274
 4                    1       8182 2020-11-22        2020-12-02                    2334
 5                    1       1732 2020-07-07        2020-07-17                    2393
 6                    1       1753 2020-09-02        2020-09-12                    2163
 7                    1       2247 2020-12-16        2020-12-26                    2530
 8                    1       7534 2020-07-05        2020-07-15                    2455
 9                    1       9378 2021-07-24        2021-08-03                    2608
10                    1        209 2021-04-25        2021-05-05                    2133
# i more rows
```

addPriorObservation()

Podemos añadir la **historia previa (fecha)** en base al periodo de observación correspondiente.

```
> cdm$curso_omop_covid |> addPriorObservation(priorObservationType = "date")
# Source:   table<og_005_1741339529> [?? x 5]
# Database: DuckDB v1.1.3 [apalomar@Windows 10 x64:R 4.4.1/C:\Users\apalomar\AppData\Local
\Temp\RtmpSsdhuU\file1e5c18635962.duckdb]
   cohort_definition_id subject_id cohort_start_date cohort_end_date prior_observation
                  <int>      <dbl> <date>            <date>          <date>
 1                    1        531 2021-09-07        2021-09-17      2013-10-04
 2                    1       6285 2020-11-04        2020-11-14      2013-12-22
 3                    1       8096 2021-01-07        2021-01-17      2014-10-17
 4                    1       8182 2020-11-22        2020-12-02      2014-07-03
 5                    1       1732 2020-07-07        2020-07-17      2013-12-18
 6                    1       1753 2020-09-02        2020-09-12      2014-10-01
 7                    1       2247 2020-12-16        2020-12-26      2014-01-12
 8                    1       7534 2020-07-05        2020-07-15      2013-10-15
 9                    1       9378 2021-07-24        2021-08-03      2014-06-03
10                    1        209 2021-04-25        2021-05-05      2015-06-23
# i more rows
```

addDemographics()

También podemos utilizar las tres funciones que acabamos de ver a la vez.

```
cdm$curs_omop_covid |>
  addAge() |>
  addSex() |>
  addPriorObservation()
```

addDemographics()

O podemos usar la función *addDemographics()*

```
addDemographics(
  x,
  indexDate = "cohort_start_date",
  age = TRUE,
  ageName = "age",
  ageMissingMonth = 1,
  ageMissingDay = 1,
  ageImposeMonth = FALSE,
  ageImposeDay = FALSE,
  ageGroup = NULL,
  missingAgeGroupValue = "None",
  sex = TRUE,
  sexName = "sex",
  missingSexValue = "None",
  priorObservation = TRUE,
  priorObservationName = "prior_observation",
  priorObservationType = "days",
  futureObservation = TRUE,
  futureObservationName = "future_observation",
  futureObservationType = "days",
  dateOfBirth = FALSE,
  dateOfBirthName = "date_of_birth",
  name = NULL
)
```

Argumentos para observación **futura**

Si queremos ver la **fecha de nacimiento**

addDemographics()

```
> cdm$curso_omop_covid |> addDemographics(dateOfBirth = TRUE)
# Source:   table<og_008_1741339821> [?? x 9]
# Database: DuckDB v1.1.3 [apalomar@Windows 10 x64:R 4.4.1/C:\Users\apalomar\AppData\Local\Temp\RtmpSsdhuU\file1e5c18635962.duckdb]
   cohort_definition_id subject_id cohort_start_date cohort_end_date   age sex    prior_observation future_observation date_of_birth
                  <int>      <dbl> <date>            <date>          <int> <chr>              <int>              <int> <date>
 1                    2          8 2021-01-08        2021-01-18          2 Male                872                563 2018-08-20
 2                    2         36 2020-10-09        2020-10-19          4 Male               1472                705 2016-09-28
 3                    2         47 2020-11-26        2020-12-06         29 Female             1678                520 1991-02-16
 4                    2         52 2020-12-08        2020-12-18         55 Female             2560                744 1965-11-11
 5                    2         58 2020-07-12        2020-07-22         53 Female             1980                953 1967-01-16
 6                    2         72 2020-12-02        2020-12-12         76 Male               2652                687 1944-05-11
 7                    2         89 2020-11-27        2020-12-07         20 Male               2491                463 2000-01-01
 8                    2        106 2020-08-16        2020-08-26         90 Male               3847                857 1930-04-23
 9                    2        137 2020-09-08        2020-09-18          7 Male               2584                916 2013-03-04
10                    2        156 2021-01-02        2021-01-12         46 Male               2228                712 1974-11-21
# i more rows
```

PatientProfiles

PatientProfiles 1.5.0 Reference Articles Changelog Search

PatientProfiles
Adding patient demographics
Adding cohort intersections
Adding concept intersections
Adding table intersections
Summarise result

PatientProfiles

CRAN 1.5.0 R-CMD-check passing lifec ownloads 42K

Package overview

PatientProfiles contains functions for adding characteristics to OMOP CDM tables containing patient level data (e.g. condition occurrence, drug exposure, and so on) and OMOP CDM cohort tables. The characteristics that can be added include an individual´s sex, age, and days of prior observation Time varying characteristics, such as age, can be estimated relative to any date in the corresponding table. In addition, PatientProfiles also provides functionality for identifying intersections between a cohort table and OMOP CDM tables containing patient level data or other cohort tables.

Package installation

You can install the latest version of PatientProfiles like so:

```
install.packages("PatientProfiles")
```

Links

View on CRAN

Browse source code

Report a bug

License

Full license

Apache License (>= 2)

Community

Contributing guide

Citation

Citing PatientProfiles

Developers

Martí Català
Author, maintainer

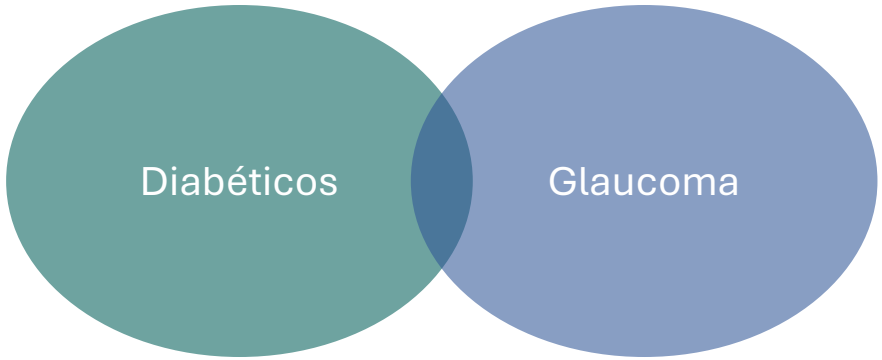
Yuchen Guo
Author

https://darwin-eu.github.io/PatientProfiles/

Add cohort intersections

Otra característica del paquete es la de poder ver de forma sencilla la **intersección entre dos cohortes de individuos en el CDM**.

Por ejemplo, queremos ver cuántas personas de una cohorte especifica también aparecen en otra cohorte (de condiciones o medicamentos, por ejemplo), en una determinada ventana temporal.



Add cohort intersections

Hay **3 funciones principales**

- addCohortIntersectFlag()
- addCohortIntersectDate()
- addCohortIntersectCount()
- addCohortIntersectDays()

addCohortIntersectFlag()

Esta función crea una nueva columna indicando si el individuo en la cohorte 1 (ej. diabetes) tiene (1) o no tiene (0) también una condición en la cohorte 2 (ej. glaucoma).

addCohortIntersectFlag()

La función tiene más opciones

```
addCohortIntersectFlag(
  x,
  targetCohortTable,
  targetCohortId = NULL,
  indexDate = "cohort_start_date",
  censorDate = NULL,
  targetStartDate = "cohort_start_date",
  targetEndDate = "cohort_end_date",
  window = list(c(0, Inf)),
  nameStyle = "{cohort_name}_{window_name}",
  name = NULL
)
```

- Tabla con individuos en el **cdm**
- **Nombre tabla** con la que queremos ver **intersección**
- **Id** de la cohorte a incluir
- Variable en x que contiene la **data** para calcular la intersección
- Si queremos censurar eventos en una fecha
- **Ventana temporal** en que considerar intersecciones

Por ejemplo, se puede definir una, o varias, ventanas temporales.

addCohortIntersectFlag()

```
> cdm$covid |> addCohortIntersectFlag(targetCohortTable = "cvd", targetCohortId = 1, window
= list(c(0,Inf))) |> filter(heartf_0_to_inf==1)
# Source:   SQL [?? x 5]
# Database: DuckDB v1.3.2 [apalomar@Windows 10 x64:R 4.4.1/C:\Users\apalomar\AppData\Local
\Temp\RtmpSUzkQg\file5dc44dd01bd.duckdb]
   cohort_definition_id subject_id cohort_start_date cohort_end_date heartf_0_to_inf
                  <int>      <int> <date>            <date>                    <dbl>
 1                    1       6448 2021-03-01        2021-03-01                    1
 2                    1       6122 2021-01-24        2021-01-24                    1
 3                    1       8791 2020-09-13        2020-09-13                    1
 4                    1       1221 2020-08-08        2020-08-08                    1
 5                    1       2609 2020-11-20        2020-11-20                    1
 6                    1       3444 2020-11-23        2020-11-23                    1
 7                    1       8622 2020-12-06        2020-12-06                    1
 8                    1       9694 2020-12-11        2020-12-11                    1
 9                    1       5548 2021-02-27        2021-02-27                    1
10                    1        819 2020-07-12        2020-07-12                    1
11                    1       7783 2020-12-16        2020-12-16                    1
12                    1       3541 2020-04-26        2020-04-26                    1
13                    1       1681 2020-11-25        2020-11-25                    1
14                    1       7569 2020-04-11        2020-04-11                    1
15                    1       3705 2020-03-15        2020-03-15                    1
16                    1      10236 2021-03-11        2021-03-11                    1
17                    1       1191 2020-12-31        2020-12-31                    1
```

addCohortIntersectDate()

IDIAP JORDI GOL 25 ANYS

También se puede mostrar la **fecha de la intersección**.

```
addCohortIntersectDate(
  x,
  targetCohortTable,
  targetCohortId = NULL,
  indexDate = "cohort_start_date",
  censorDate = NULL,
  targetDate = "cohort_start_date",
  order = "first",
  window = c(0, Inf),
  nameStyle = "{cohort_name}_{window_name}",
  name = NULL
)
```

- Tabla con individuos en el **cdm**
- **Nombre tabla** con la que queremos ver **intersección**
- **Id** de la cohorte a incluir
- Variable en x que contiene la **data** para calcular la intersección
- Si queremos censurar eventos en una fecha
- Fecha que utilizar si existen **múltiples registros** para un individuo dentro de la ventana de interés. Puede ser la **primera** (first) o la **última** (last)
- **Ventana temporal** en que considerar intersecciones

Por ejemplo, se puede definir una, o varias, ventanas temporales.

addCohortIntersectCount ()

25ANYS IDIAP JORDI GOL

También se pueden **mostrar el número de veces que se interseccionan.**

```
addCohortIntersectCount(
  x,
  targetCohortTable,
  targetCohortId = NULL,
  indexDate = "cohort_start_date",
  censorDate = NULL,
  targetStartDate = "cohort_start_date",
  targetEndDate = "cohort_end_date",
  window = list(c(0, Inf)),
  nameStyle = "{cohort_name}_{window_name}",
  name = NULL
)
```

- Tabla con individuos en el **cdm**
- **Nombre tabla** con la que queremos ver **intersección**
- **Id** de la cohorte a incluir
- Variable en x que contiene la **data** para calcular la intersección
- Si queremos censurar eventos en una fecha
- **Ventana temporal** en que considerar intersecciones

addCohortIntersectDays()

Calcula el número de días hasta un evento de una cohorte de referencia (target cohort).

```
addCohortIntersectDays(
  x,
  targetCohortTable,
  targetCohortId = NULL,
  indexDate = "cohort_start_date",
  censorDate = NULL,
  targetDate = "cohort_start_date",
  order = "first",
  window = c(0, Inf),
  nameStyle = "{cohort_name}_{window_name}",
  name = NULL
)
```

- Tabla con individuos en el **cdm**
- **Nombre tabla** con la que queremos ver **intersección**
- **Id** de la cohorte a incluir
- Variable en x que contiene la **data** para calcular la intersección
- Si queremos censurar eventos en una fecha
- Fecha que usar si hay múltiples registros para un mismo individuo durante la ventana temporal . “first” or “last”
- **Ventana temporal** en que considerar intersecciones

PatientProfiles

PatientProfiles 1.5.0 Reference Articles Changelog Search

PatientProfiles
Adding patient demographics
Adding cohort intersections
Adding concept intersections
Adding table intersections
Summarise result

PatientProfiles

CRAN 1.5.0 | R-CMD-check passing | lifec | ownloads 42K

Package overview

PatientProfiles contains functions for adding characteristics to OMOP CDM tables containing patient level data (e.g. condition occurrence, drug exposure, and so on) and OMOP CDM cohort tables. The characteristics that can be added include an individual´s sex, age, and days of prior observation Time varying characteristics, such as age, can be estimated relative to any date in the corresponding table. In addition, PatientProfiles also provides functionality for identifying intersections between a cohort table and OMOP CDM tables containing patient level data or other cohort tables.

Package installation

You can install the latest version of PatientProfiles like so:

```
install.packages("PatientProfiles")
```

Links

View on CRAN

Browse source code

Report a bug

License

Full license

Apache License (>= 2)

Community

Contributing guide

Citation

Citing PatientProfiles

Developers

Martí Català
Author, maintainer

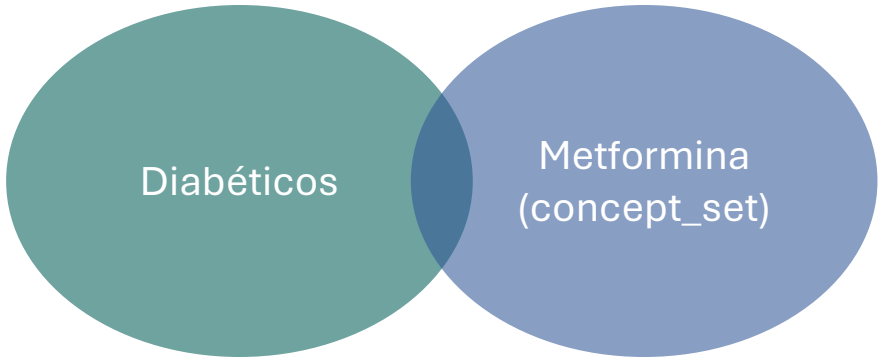
Yuchen Guo
Author

https://darwin-eu.github.io/PatientProfiles/

Add concept intersections

Otra característica del paquete es la de poder ver de forma sencilla la **intersección entre una cohorte de individuos en el CDM y un concept set.**

Por ejemplo, queremos ver cuántas personas de una cohorte especifica tiene un registro de un determinado concept set (de condiciones o medicamentos, por ejemplo), en una determinada ventana temporal.



Add concept intersections

Interescción con...	Cohortes	Concept Sets
Origen de datos	Registros procesados y colapsados	Datos brutos (nivel de registro)
Requisitos	Filtra por periodo de observación	Sin filtros automáticos de observación
Flujo	Requiere crear cohorte previa	Adición directa de variables

Add concept intersections

Hay **3 funciones principales**

- addConceptIntersectFlag()
- addConceptIntersectDate()
- addConceptIntersectCount()
- addConceptIntersectDays()

PatientProfiles

PatientProfiles 1.5.0 Reference Articles Changelog Search

PatientProfiles
Adding patient demographics
Adding cohort intersections
Adding concept intersections
Adding table intersections
Summarise result

PatientProfiles

CRAN 1.5.0 | R-CMD-check passing | lifec | ownloads 42K

Package overview

PatientProfiles contains functions for adding characteristics to OMOP CDM tables containing patient level data (e.g. condition occurrence, drug exposure, and so on) and OMOP CDM cohort tables. The characteristics that can be added include an individual´s sex, age, and days of prior observation Time varying characteristics, such as age, can be estimated relative to any date in the corresponding table. In addition, PatientProfiles also provides functionality for identifying intersections between a cohort table and OMOP CDM tables containing patient level data or other cohort tables.

Package installation

You can install the latest version of PatientProfiles like so:

```
install.packages("PatientProfiles")
```

Links

View on CRAN

Browse source code

Report a bug

License

Full license

Apache License (>= 2)

Community

Contributing guide

Citation

Citing PatientProfiles

Developers

Martí Català
Author, maintainer

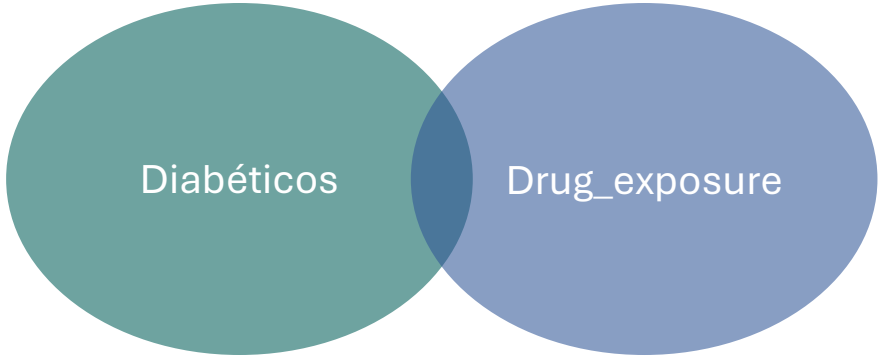
Yuchen Guo
Author

https://darwin-eu.github.io/PatientProfiles/

Add table intersections

Hasta ahora hemos visto que podemos añadir variables de intersección basadas en cohortes o *concept sets*.

Una opción adicional que tenemos es, simplemente, añadir una **intersección basada en una tabla del CDM** (condition_ocurrence, visit_ocurrence, drug_exposure...).



Iniciación práctica al análisis de datos OMOP

CohortCharacteristics

25ANYS IDIAP JORDI GOL

Objetivo

- El objetivo del paquete ***CohortCharacteristics*** es resumir y describir características generales de una cohorte de pacientes.

- Calcula **estadísticas descriptivas a nivel de cohorte**, como edad media, distribución de género, comorbilidades, etc.

CohortCharacteristics

- Se ha desarrollado para el **Proyecto de Darwin EU®** por el equipo OxInfer.

- Está disponible en CRAN y se puede instalar fácilmente en R.

- ¿Cómo podemos **descargarlo**?

```
install.packages("CohortCharacteristis")
library(CohortCharacteristics)
```

https://CRAN.R-project.org/package=CohortCharacteristics

CohortCharacteristics

CohortCharacteristics 1.1.1 Reference Articles Changelog Search

- Summarise cohort entries
- Summarise patient characteristics
- Summarise large scale characteristics
- Summarise cohort overlap
- Summarise cohort timing

CohortCharacterist

CRAN 1.1.1 codecov 95% R-CMD-check passi

Package overview

CohortCharacteristics contains functions for summarising characteristics of cohorts of patients identified in an OMOP CDM dataset. Once a cohort table has been created, CohortCharacteristics provides a number of functions to help provide a summary of the characteristics of the individuals within the cohort.

```
#> To cite package 'CohortCharacteristics' in publications use:
#>
#>   Catala M, Guo Y, Lopez-Guell K, Burn E, Mercade-Besora N, Alcalde M
#>   (????). _CohortCharacteristics: Summarise and Visualise
#>   Characteristics of Patients in the OMOP CDM_. R package version
#>   1.0.1, <https://darwin-eu.github.io/CohortCharacteristics/>.
#>
#> A BibTeX entry for LaTeX users is
```

Links

View on CRAN

Browse source code

Report a bug

License

Full license

Apache License (>= 2)

Community

Contributing guide

Citation

Citing CohortCharacteristics

Developers

Martí Català

Author, maintainer

https://darwin-eu.github.io/CohortCharacteristics/

Funciones CohortCharacteristics

summariseCharacteristics 14
summariseCohortAttrition 17
summariseCohortCount 18
summariseCohortOverlap 18
summariseCohortTiming 19
summariseLargeScaleCharacteristics 20

Resumir información

tableCharacteristics 22
tableCohortAttrition 23
tableCohortCount 24
tableCohortOverlap 25
tableCohortTiming 26
tableLargeScaleCharacteristics 27

Crear tablas en base a los datos resumidos

plotCharacteristics 6
plotCohortAttrition 7
plotCohortCount 8
plotCohortOverlap 9
plotCohortTiming 10
plotComparedLargeScaleCharacteristics 12
plotLargeScaleCharacteristics 13

Representar gráficamente estos datos

summariseCharacteristics

- Resumir características en una cohorte.

```
summariseCharacteristics(
  cohort,
  cohortId = NULL,
  strata = list(),
  counts = TRUE,
  demographics = TRUE,
  ageGroup = NULL,
  tableIntersectFlag = list(),
  tableIntersectCount = list(),
  tableIntersectDate = list(),
  tableIntersectDays = list(),
  cohortIntersectFlag = list(),
  cohortIntersectCount = list(),
  cohortIntersectDate = list(),
  cohortIntersectDays = list(),
  conceptIntersectFlag = list(),
  conceptIntersectCount = list(),
  conceptIntersectDate = list(),
  conceptIntersectDays = list(),
  otherVariables = character(),
  otherVariablesEstimates = c("min", "q25", "median", "q75", "max",
                              "percentage")
)
```

Tabla con individuos en el **cdm**

Id de la cohorte

Lista de variables para estratificar los resultados. Tienen que añadirse previamente

Lista de grupos de edad para estratificar los resultados.

Si queremos mostrar intersecciones con alguna tabla, cohorte o concept set

Si quieres resumir otras variables contenidas en la cohorte.

Para modificar los estimates por defecto

tableCharacteristics

- Formatear un objeto summarised_characteristics en una tabla visual.

```
tableCharacteristics(
  result,
  type = NULL,
  header = c("cdm_name", "cohort_name"),
  groupColumn = character(),
  hide = c(additionalColumns(result), settingsColumns(result)),
  style = NULL,
  .options = list()
)
```

Objeto summarised_characteristics

Formatos tabla “gt”, “flextable”, “tibble”...

Vector con los elementos en orden para encabezado

Un vector indicando los grupos a dividir

- Se puede exportar fácilmente a Word

```
table |> gt::gtsave("table.docx")
```

tableCharacteristics

```
cdm$covid |>
  addSex() |>
  summariseCharacteristics(strata = list("sex")) |>
  tableCharacteristics(header = "sex", hide = c("cohort_name", "cdm_name"))
```

Variable name	Variable level	Estimate name	Table name	Sex		
				overall	Female	Male
Number records	–	N	temp	964	497	467
Number subjects	–	N	temp	964	497	467
Cohort start date	–	Median [Q25 - Q75]	temp	2020-12-13 [2020-10-22 - 2021-01-31]	2020-12-17 [2020-10-19 - 2021-02-01]	2020-12-09 [2020-10-27 - 2021-01-31]
		Range	temp	2020-03-15 to 2021-09-14	2020-03-16 to 2021-09-14	2020-03-15 to 2021-09-08
Cohort end date	–	Median [Q25 - Q75]	temp	2021-01-05 [2020-11-11 - 2021-02-25]	2021-01-07 [2020-11-08 - 2021-02-25]	2021-01-02 [2020-11-15 - 2021-02-24]
		Range	temp	2020-03-15 to 2021-10-12	2020-04-01 to 2021-10-12	2020-03-15 to 2021-09-27
Age	–	Median [Q25 - Q75]	temp	45 [20 - 72]	44 [20 - 69]	46 [20 - 76]
		Mean (SD)	temp	45.97 (29.46)	44.85 (29.00)	47.16 (29.91)
		Range	temp	0 to 97	0 to 97	0 to 97

plotCharacteristics

- Crea un ggplot a partir de la salida de summariseCharacteristics.

```
plotCharacteristics(
  data,
  x = "variable_name",
  plotStyle = "barplot",
  facet = NULL,
  colour = NULL,
  colourName = NULL,
  .options = list()
)
```

Output de **summariseCharacteristics**

Qué poner en **eje X**. Tiene que ser una columna en los datos

Tipo de **gráfico**. Solo boxplot o barplot

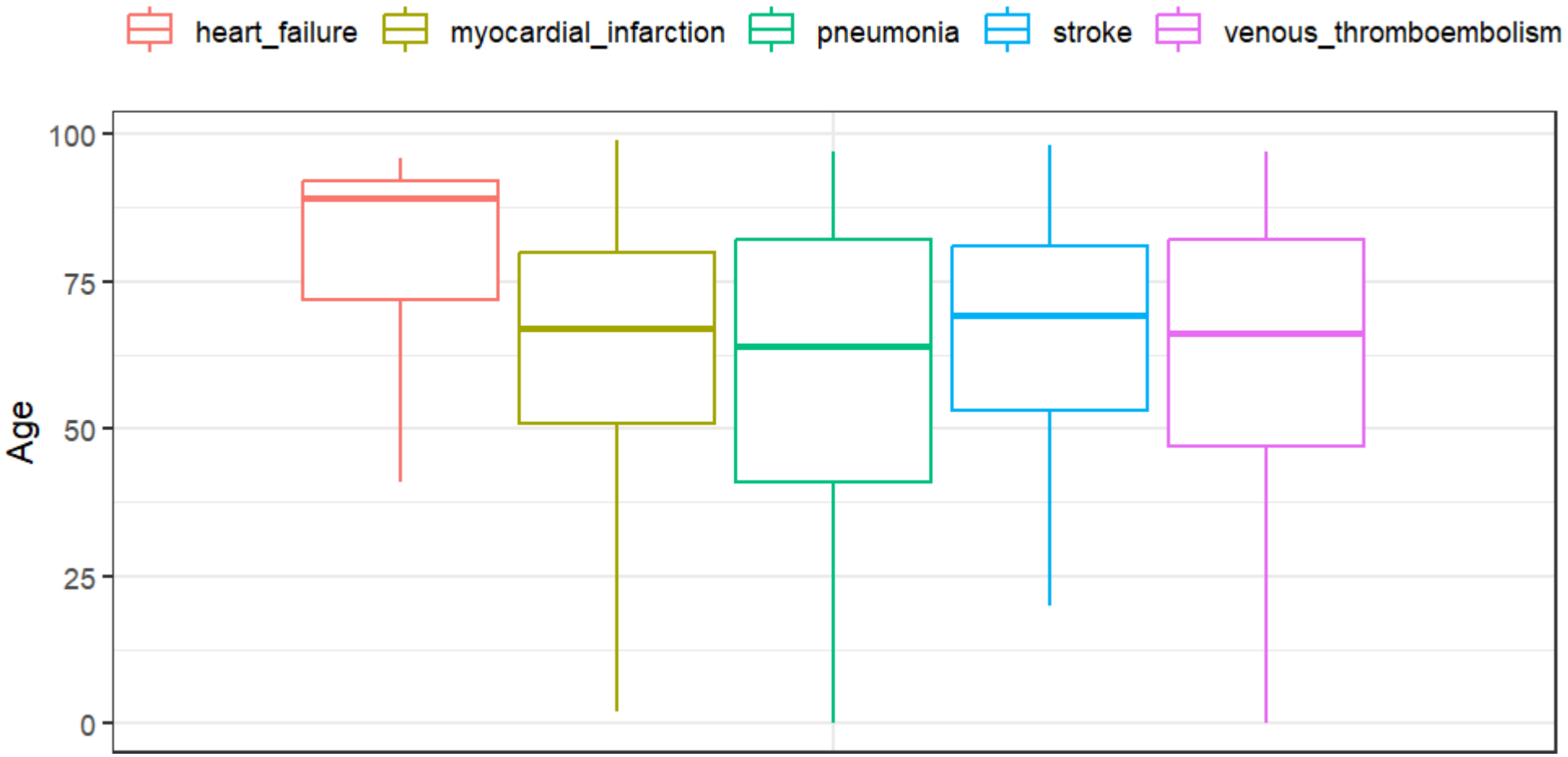
Si queremos **separar** por alguna variable

Si queremos darle **color** por una variable

Nombre para la leyenda de color

Opciones adicionales

plotCharacteristics




```
cdm$condiciones |> summariseCharacteristics() |> filter(variable_name=="Age") |>
  plotCharacteristics(plotStyle = "boxplot", colour = "group_level")
```

summariseLargeScaleCharacteristics

- Esta función se utiliza para resumir las **características a gran escala** de una tabla de cohorte, proporcionando un **resumen descriptivo detallado** de las características **clínicas** y de **tratamiento**.

```
summariseLargeScaleCharacteristics(
  cohort,
  strata = list(),
  window = list(c(-Inf, -366), c(-365, -31), c(-30, -1), c(0, 0), c(1, 30), c(31, 365),
                c(366, Inf)),
  eventInWindow = NULL,
  episodeInWindow = NULL,
  indexDate = "cohort_start_date",
  censorDate = NULL,
  includeSource = FALSE,
  minimumFrequency = 0.005,
  excludedCodes = c(0)
)
```

- `cohort` → Cohorte a caracterizar
- `strata` → Lista de estratificación
- `window` → Ventana(s) temporales a caracterizar
- `eventInWindow` → Tabla a caracterizar los eventos (ej. condiciones)
- `episodeInWindow` → Tabla a caracterizar los episodios (fármacos)
- `includeSource` → Si queremos ver los source concept
- `minimumFrequency` → Umbral de frecuencia mínima

tableLargeScaleCharacteristics

- Formatear un objeto summarised_characteristics en una tabla visual.

```
tableLargeScaleCharacteristics(
  result,
  compareBy = NULL,
  hide = c("type"),
  smdReference = NULL,
  type = "reactable"
)
```

Columna de comparación

Nivel de referencia para la Diferencia Media Estandarizada (SMD); debe ser uno de los valores presentes en la columna compareBy.

```
tableTopLargeScaleCharacteristics(
  result,
  topConcepts = 10,
  type = NULL,
  style = NULL
)
```

tableTopLargeScaleCharacteristics

	Window			
	-365 to -1		1 to 365	
	Table name			
	condition_occurrence	drug_exposure	condition_occurrence	drug_exposure
	Type			
Top	event	episode	event	episode
1	Fever (437663) 51 (5.3%)	Influenza, seasonal, injectable, preservative free (40213154) 759 (78.7%)	Acute deep venous thrombosis (44782746) 60 (6.3%)	Influenza, seasonal, injectable, preservative free (40213154) 738 (77.5%)
2	Cough (254761) 43 (4.5%)	tetanus and diphtheria toxoids, adsorbed, preservative free, for adult use (5 Lf of tetanus toxoid and 2 Lf of diphtheria toxoid) (40213227) 66 (6.8%)	Acute pulmonary embolism (45768439) 58 (6.1%)	acetaminophen 500 MG Oral Tablet (19020053) 164 (17.2%)
3	Loss of taste (4289517) 26 (2.7%)	diphtheria, tetanus toxoids and acellular pertussis vaccine (40213281) 52 (5.4%)	Acute respiratory distress syndrome (4195694) 29 (3.0%)	0.4 ML enoxaparin sodium 100 MG/ML Prefilled Syringe (40160973) 152 (16.0%)
4	Fatigue (4223659) 19 (2.0%)	poliovirus vaccine, inactivated (40213160) 45 (4.7%)	Heart failure (316139) 17 (1.8%)	1 ML enoxaparin sodium 150 MG/ML Prefilled Syringe (40160997) 122 (12.8%)
5	Sputum finding (4089228) 17 (1.8%)	pneumococcal conjugate vaccine, 13 valent (40213198) 37 (3.8%)	Injury of kidney (4220631) 16 (1.7%)	tetanus and diphtheria toxoids, adsorbed, preservative free, for adult use (5 Lf of tetanus toxoid and 2 Lf of diphtheria toxoid) (40213227) 69 (7.2%)
6	Dyspnea (312437) 14 (1.4%)	measles, mumps and rubella virus vaccine (40213183) 32 (3.3%)	Acquired coagulation disorder (4120613) 15 (1.6%)	NDA020503 200 ACTUAT albuterol 0.09 MG/ACTUAT Metered Dose Inhaler (1511248) 64 (6.7%)

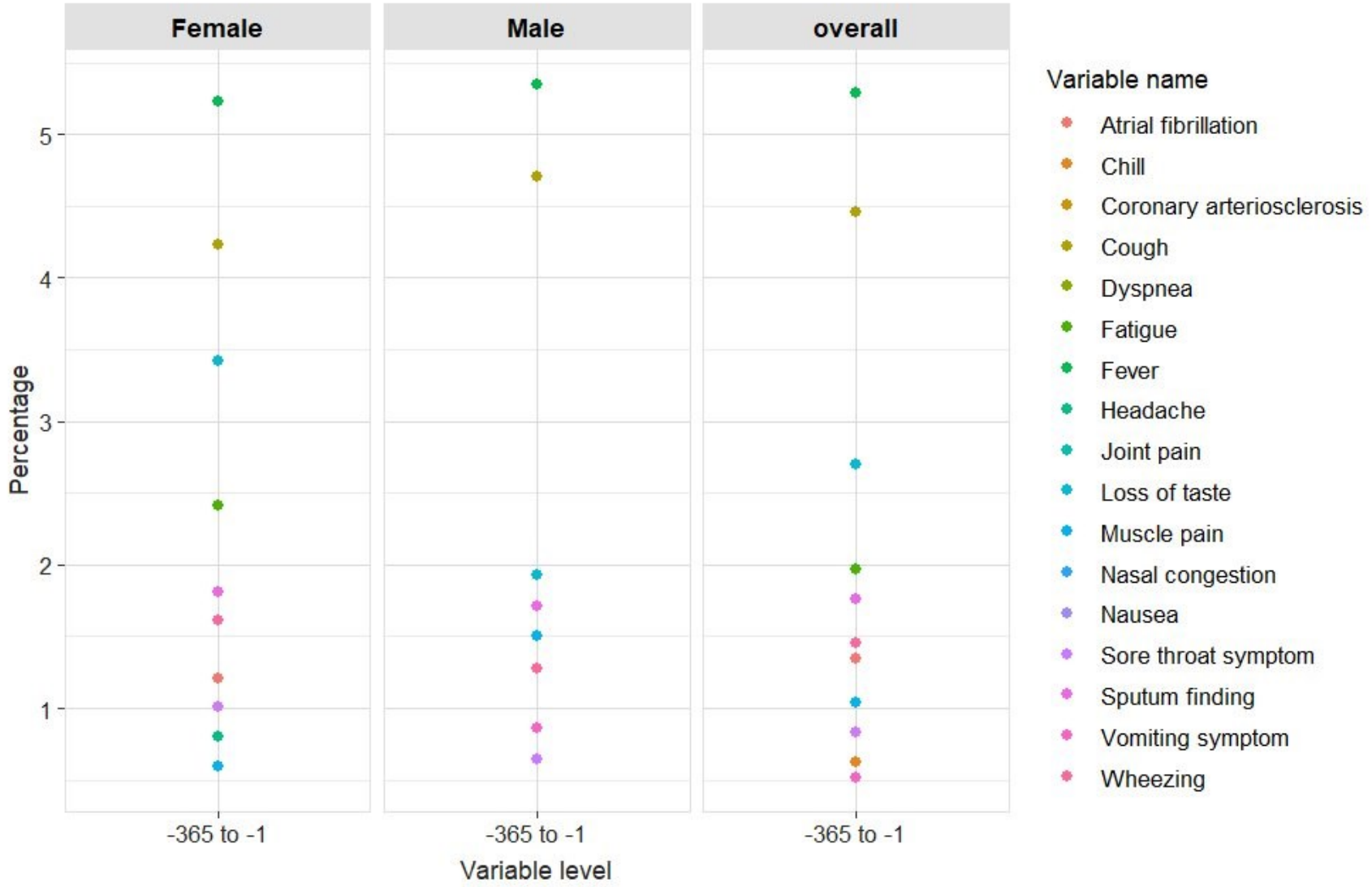
plotLargeScaleCharacteristics

- Crea un ggplot a partir de la salida de summariseLargeScaleCharacteristics.

```
plotLargeScaleCharacteristics(
  result,
  facet = c("cdm_name", "cohort_name"),
  colour = "variable_level",
  style = NULL
)
```

```
plot <- cdm$covid |>
  addSex() |>
  summariseLargeScaleCharacteristics(eventInWindow = "condition_occurrence",
                                     window = list(c(-365, -1)),
                                     strata = list("sex")) |>
  plotLargeScaleCharacteristics(colour = "variable_name", facet = "sex")
plot
```

plotLargeScaleCharacteristics



Práctica

- Cohortes
- 1_PatientProfiles
- PatientProfiles
- patientprofiles_practica
- patientprofiles_soluciones

Objetivo

El objetivo de esta práctica es caracterizar una población a partir de varias tablas de una base de datos en formato OMOP. Veremos cómo añadir de forma sencilla información demográfica, combinar tablas, seleccionar según criterios (flowchart) y hacer una tabla descriptiva.

¿Cómo funciona?

En este fichero encontrarás una serie de ejercicios que te proponemos, acompañados de teoría y pistas para su resolución. Puedes crear un script.R en el mismo directorio donde está este fichero, e ir resolviéndolos en el script. En cada ejercicio hay pistas, puesto que el curso es abierto a muchos perfiles y habrá gente que necesitará indicaciones sobre programación en R, otras sobre dónde encontrar las cosas en OMOP, y otras con todo. Además, en este mismo directorio, hay un fichero adicional con las soluciones a los ejercicios, para que puedas autocorregirte la práctica. Si tienes cualquier duda, pregunta a los docentes de apoyo en las prácticas, ¡e intenta utilizar las soluciones solo para corregir!